

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering CO1 ASSESSMENT TOOL 1 – Case Study Analysis

Student Name: Rohith Krishna G

Reg. No.: 192524079

Course Code: CSA1010

Code of Conduct

I **Rohith Krishna G (192524079)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Case Study Analysis

1. Understanding the Problem Statement

1.1 The Problem in My Own Words

Every organization that sells a product or provides a service receives complaints and support requests. In many organizations these arrive through scattered channels such as phone calls, e-mails, walk-in counters, social-media messages and handwritten registers. Because there is no single place where all of them are recorded, complaints are often lost, duplicated, assigned to the wrong person, or forgotten. The customer, who is already unhappy, has no way of knowing whether the complaint was received, who is working on it, or when it will be solved, and is forced to call repeatedly to ask for an update. The organization, on the other hand, cannot measure how many complaints are pending, how long they take to close, or which problems keep repeating.

The problem is therefore to design and develop a **centralized, web-based helpdesk system** in which a customer can register a complaint once, receive a unique ticket number, follow the progress of that ticket at any time, exchange messages with the support staff handling it, and be notified automatically at every important step until the complaint is fully resolved and closed. At the same time, the system must give support agents and managers a structured way to receive, assign, prioritize, resolve and analyse complaints.

1.2 Objectives of the Proposed System

- **Centralised complaint capture:** provide a single, easy-to-use channel (web portal, mobile-friendly interface, optional e-mail intake) where customers can submit complaints with a description, category, priority and attachments such as screenshots, invoices or photos.
- **Unique ticket generation:** automatically create a ticket with a unique ID, timestamp and initial status so that every complaint can be tracked individually.
- **Real-time status tracking:** allow customers to see the current status of a ticket (New, Assigned, In Progress, Pending Customer, Escalated, Resolved, Closed) and its complete history.
- **Two-way communication:** enable threaded conversation between the customer and the assigned support staff inside the ticket, so that all communication is stored in one place.
- **Automated notifications:** send e-mail, SMS and in-app notifications on ticket creation, assignment, every reply, status change, escalation, resolution and closure – until the complaint is resolved.
- **Efficient allocation and escalation:** route tickets to the right agent or team based on category, workload and priority, and escalate automatically when the service-level agreement (SLA) time is about to be crossed.
- **Transparency and accountability:** maintain an audit trail of who did what and when on every ticket.
- **Reporting and continuous improvement:** provide dashboards and reports (open tickets, average resolution time, SLA compliance, repeat issues, customer satisfaction) to help management improve products and services.

1.3 Scope of the Problem

In scope – the following functions will be covered by the proposed system:

- Customer registration, login and profile management.
- Complaint (ticket) submission with categories, priority and file attachments.

- Ticket tracking, search and history for customers; ticket queue and workbench for agents.
- Assignment, re-assignment, escalation and SLA monitoring.
- In-ticket messaging between customer and support staff, including internal notes visible only to staff.
- Multi-channel notifications (e-mail, SMS, in-app) and reminders.
- Customer feedback/rating after resolution, reopening of tickets within a defined period.
- Administration (users, roles, categories, SLA rules, templates) and reports/dashboards.

Out of scope – the first release will not include: the actual repair/replacement logistics of the product, billing and payment processing, a full CRM or sales module, voice-call (IVR) telephony integration, and AI-based automatic resolution. These are listed as future enhancements.

Boundaries and assumptions: the users are customers of a single organization (a multi-tenant version is a future extension); customers have internet access and a valid e-mail ID or mobile number; third-party e-mail and SMS gateways are available; and the organization will define its own complaint categories and SLA times.

2. Key Stakeholders and Challenges

2.1 Stakeholders, Roles and Responsibilities

Stakeholder	Role in the System	Responsibilities
Customer (Complainant)	Primary user who raises the complaint	Register/log in and submit complaints with correct details and evidence. Track ticket status, reply to agent queries, confirm resolution or reopen, and give feedback.
Support Agent	Front-line staff who handle tickets	Accept assigned tickets, investigate, update status, reply to customers and add internal notes. Resolve within SLA or escalate when the issue is beyond their level.
Support Team Lead / Supervisor	Manages the support team	Monitor queues and workloads, assign/re-assign tickets, handle escalations. Ensure SLA compliance and quality of responses.
Department / Technical Experts (L2/L3)	Specialists in billing, product, logistics, IT, etc.	Provide technical or domain solutions for escalated tickets. Update the agent with findings and fixes.
System Administrator	Configures and maintains the system	Manage user accounts, roles and permissions, categories, SLA rules and notification templates. Take backups, monitor performance and security.
Management / Business Owner	Decision maker and sponsor	Define service policies and targets, review dashboards and reports. Use complaint trends to improve products, processes and customer satisfaction.
Development & Maintenance Team	Builds and supports the software	Analyse requirements, design, code, test, deploy, and fix defects. Deliver enhancements and keep the system secure and up to date.
External Service Providers	E-mail, SMS and cloud/hosting vendors	Deliver notifications reliably and keep the infrastructure available.

Stakeholder	Role in the System	Responsibilities
Regulatory / Compliance Authority	Legal and data-protection oversight	Ensure that customer data is stored, used and retained as per data-protection and consumer-protection laws.

2.2 Major Challenges and Issues Faced by the Stakeholders

Stakeholder	Major Challenges / Issues
Customer	No acknowledgement or ticket number after complaining. No visibility of progress; has to call or e-mail again and again. Must repeat the same story to different agents; delayed or no response; no notification when the issue is resolved.
Support Agent	Complaints arrive from many channels and get scattered; difficulty in finding the history of a customer. Unequal workload, unclear priorities, repeated manual updates and incomplete information from customers.
Team Lead / Supervisor	No real-time view of pending and overdue tickets; hard to balance workload. Escalations are noticed late; manual preparation of performance reports.
Technical / Department Experts	Receive escalations without enough details or logs; poor coordination with front-line staff.
System Administrator	Managing roles and access, ensuring availability during peak load, and protecting sensitive data from misuse or breach.
Management	Lack of reliable data on complaint volume, resolution time and recurring defects; falling customer satisfaction and loyalty; reputation damage and customer churn.
Development Team	Changing requirements, integration with e-mail/SMS gateways, performance and security needs, and keeping the system maintainable.

Cross-cutting challenges: (i) high complaint volumes during product launches or service outages; (ii) maintaining a consistent response time (SLA) across all agents; (iii) data privacy and confidentiality of customer information; (iv) multiple languages and customers with different technical skills; and (v) ensuring that notifications are timely but not excessive.

3. Analysis of the Current Approach

3.1 Description of the Existing System / Current Process

In most small and medium organizations, complaints are handled by a largely manual or semi-manual process. The typical flow is:

- The customer telephones the helpline, visits the service counter, sends an e-mail, or posts on social media.
- A staff member notes the complaint in a register, spreadsheet or personal mailbox, and sometimes forwards it verbally or by e-mail to a colleague or department.
- The responsible person investigates, and informs the customer by phone or e-mail if and when time permits.
- Follow-up depends on the customer calling back; there is no automatic reminder, and closure is rarely confirmed formally.
- Management occasionally compiles data manually from registers or spreadsheets to prepare monthly reports.

3.2 Strengths of the Current Approach

- **Low initial cost and simplicity:** no software development or training is needed.
- **Personal touch:** phone calls and walk-ins allow human empathy and quick clarification for simple problems.
- **Flexibility:** staff can handle unusual cases informally without being restricted by a rigid workflow.
- **Familiarity:** employees and customers are already used to the channels.

3.3 Limitations of the Current Approach

- **No central record:** information is spread across registers, spreadsheets and mailboxes, so complaints are lost or duplicated.
- **No tracking or transparency:** customers cannot see the status; staff cannot easily find history.
- **Delays and no SLA control:** there is no priority, deadline or automatic escalation.
- **Manual and error-prone:** handwriting errors, wrong routing, and inconsistent data.
- **Poor scalability:** the process breaks down when complaint volume grows.
- **Weak communication:** no automatic notifications; conversations are scattered across calls and e-mails.
- **Little analytics:** no reports on trends, repeat issues or agent performance.
- **Security and privacy risks:** paper records and shared spreadsheets can be accessed or leaked easily.

3.4 Problems and Gaps to Be Addressed

Gap in Current Process	Impact	What the New System Must Provide
No unique ticket / central repository	Lost, duplicate or forgotten complaints	Automatic ticket ID and a single database for all channels
No status visibility for customers	Repeated follow-up calls; frustration	Customer portal with live status and ticket history
No prioritisation or SLA	Urgent issues wait too long	Priority levels, SLA timers and automatic escalation
Fragmented communication	Miscommunication; repeated explanations	Threaded in-ticket messaging with attachments
No proactive notifications	Customer is left in the dark	E-mail/SMS/in-app alerts at every stage until closure
No reports or analytics	Management cannot improve	Dashboards and exportable reports
Weak access control and audit	Data misuse risk; no accountability	Role-based access and activity logs

4. Proposed Solution Using Software Engineering Principles

4.1 Overview of the Software-Based Solution

The proposed solution is a **web-based Customer Complaint and Support Management System (CCSMS)** with a responsive interface that works on desktop and mobile browsers. It follows a **layered, three-tier architecture** so that each layer can be developed, tested and scaled independently:

Layer	Components	Purpose
Presentation Layer	Customer Portal, Agent Workbench, Admin & Manager Dashboard (HTML5/CSS3, React/Angular, responsive design)	User interaction, forms, ticket views, chat and dashboards
Application / Business Layer	REST APIs with modules: Authentication, Ticket Management, Assignment & SLA Engine, Messaging, Notification Service, Reporting (Java Spring Boot / Node.js / Python Django)	Business rules, workflow, validation, security checks
Data Layer	Relational database (MySQL/PostgreSQL), file storage for attachments, cache (Redis) and audit-log tables	Reliable storage, fast retrieval and traceability
Integration Layer	E-mail (SMTP) gateway, SMS gateway, push/in-app notification service, optional e-mail-to-ticket parser	Delivery of notifications and external communication

Ticket life cycle (workflow): the system moves each complaint through the following states, and every transition triggers a notification and an audit-log entry:

Step	Status	What Happens	Notification Sent To
1	New	Customer submits complaint; unique ticket ID created; SLA timer starts	Customer (acknowledgement); Support Lead
2	Assigned	Ticket auto-routed or manually assigned to an agent/team based on category and workload	Agent; Customer
3	In Progress	Agent investigates, communicates with customer, adds internal notes	Customer (on each reply)
4	Pending Customer	Agent needs more information; SLA timer paused	Customer (reminder)
5	Escalated	SLA near breach or issue complex; passed to Team Lead / L2-L3 expert	Supervisor; Agent; Customer
6	Resolved	Solution provided; customer asked to confirm	Customer
7	Closed / Reopened	Customer confirms or auto-closes after a set period and gives rating; can reopen within the allowed period	Customer; Agent; Management (reports)

4.2 Major Functional Modules

No.	Module	Key Functions
1	User & Access Management	Registration, login, password reset, profile, role-based access (Customer, Agent, Supervisor, Admin, Manager), multi-factor authentication (optional)
2	Complaint Submission	Complaint form with category, sub-category, priority, description, order/product ID, attachments; auto-generated ticket ID; duplicate detection
3	Ticket Management & Tracking	Ticket list, search/filter, status updates, ticket history timeline, reopen, merge duplicates

No.	Module	Key Functions
4	Assignment, Escalation & SLA Engine	Rule-based auto-assignment, workload balancing, SLA timers, automatic escalation and reminders
5	Communication / Messaging	Threaded conversation between customer and agent, canned responses/templates, internal notes, file sharing
6	Notification Service	E-mail, SMS and in-app notifications for each event; user notification preferences; retry on failure
7	Knowledge Base / FAQ	Articles and FAQs suggested to customers before they raise a ticket to reduce avoidable complaints
8	Feedback & Satisfaction	Rating and comments after closure; CSAT score calculation
9	Reporting & Analytics	Dashboards: open/closed tickets, average response and resolution time, SLA compliance, agent performance, top complaint categories; export to PDF/Excel
10	Administration & Configuration	Manage categories, SLA policies, templates, holidays/working hours, audit logs, backup settings

4.3 Functional Requirements

ID	Requirement
FR1	The system shall allow a customer to register, log in securely and manage his/her profile.
FR2	The system shall allow a customer to submit a complaint with category, priority, description and attachments.
FR3	The system shall generate a unique ticket ID and send an acknowledgement immediately after submission.
FR4	The system shall allow customers to view the status and full history of their own tickets at any time.
FR5	The system shall automatically assign tickets to agents/teams using category and workload rules, and allow manual re-assignment by the supervisor.
FR6	The system shall allow agents to update ticket status, reply to customers and add internal notes.
FR7	The system shall support two-way messaging between customer and agent inside the ticket, with attachments.
FR8	The system shall send e-mail, SMS and in-app notifications on creation, assignment, reply, status change, escalation, resolution and closure.
FR9	The system shall track SLA deadlines and automatically escalate and alert when a deadline is about to be breached.
FR10	The system shall allow customers to confirm resolution, reopen a ticket within the allowed period and give feedback.
FR11	The system shall provide search, filter and sort options for tickets (by ID, status, date, customer, category).
FR12	The system shall generate dashboards and downloadable reports for supervisors and management.
FR13	The administrator shall be able to manage users, roles, categories, SLA rules and templates.
FR14	The system shall maintain an audit log of all actions performed on a ticket.

4.4 Non-Functional Requirements

Attribute	Requirement
Performance	Pages should load within 3 seconds; ticket search results within 2 seconds under normal load; notifications dispatched within 1 minute of an event.
Scalability	Support at least 1,000 concurrent users and growth in ticket volume through horizontal scaling and database indexing/caching.
Reliability & Availability	Availability of at least 99.5%; automatic daily backups; no loss of submitted tickets; retry mechanism for failed notifications.
Security	Encrypted communication (HTTPS/TLS); password hashing; role-based access; protection against SQL injection, XSS and CSRF; encryption of sensitive data; session timeout.
Usability	Simple, consistent and responsive interface; ticket submission in not more than 3 steps; accessibility guidelines (WCAG); multilingual support (English, Tamil, Hindi).
Maintainability	Modular, well-documented code; coding standards; at least 70% automated test coverage; easy configuration without code changes.
Portability / Compatibility	Works on major browsers (Chrome, Edge, Firefox, Safari) and mobile devices; deployable on cloud or on-premise servers.
Auditability & Compliance	All actions logged with user and time; data retention and privacy as per applicable data-protection law.

4.5 How Software Engineering Principles Are Incorporated

Principle	How It Is Incorporated in the Solution
Modularity	The system is divided into independent modules (ticketing, messaging, notification, SLA, reporting). Each has a clear interface (REST API), high cohesion and low coupling, so modules can be developed in parallel and replaced without affecting others.
Scalability	Stateless application servers behind a load balancer, database indexing, caching of frequent queries, asynchronous queue for notifications, and cloud deployment allow the system to handle increasing users and tickets.
Maintainability	Layered architecture, design patterns (MVC, Observer for notifications, Strategy for assignment rules), coding standards, code reviews, version control (Git), documentation and automated tests make defects easy to find and enhancements easy to add.
Reliability	Input validation, exception handling, transaction management, database backups and replication, retry/queue mechanism for notifications, health monitoring and logging ensure the system works correctly and recovers from failures.
Usability	Clean, responsive UI with a short complaint form, clear status indicators, search, helpful messages, FAQs, and user testing with real customers and agents to refine the design.
Security	Authentication and role-based authorization, HTTPS, hashed passwords, validation against common attacks, secure file upload scanning, audit logs and privacy-by-design for customer data.
Separation of Concerns & Abstraction	UI, business logic and data access are separated; third-party e-mail/SMS providers are hidden behind a notification interface so providers can be changed easily.

Principle	How It Is Incorporated in the Solution
Reusability	Common components (authentication, notification, logging) and ticket templates are reused across modules and can be reused in other organizational systems.
Testability	Unit, integration, system and user-acceptance testing supported by modular design; test automation is integrated in the CI pipeline.

5. Justification of the Chosen Methodology / Framework

5.1 Selected Methodology: Agile – Scrum Framework (with DevOps/CI-CD practices)

For the Customer Complaint and Support Management System, the **Agile methodology using the Scrum framework** is selected. Work is divided into short, fixed-length iterations called **sprints (2 weeks each)**, at the end of which a working, tested increment of the software is delivered. Continuous Integration/Continuous Delivery (DevOps practices) supports the Scrum team in releasing those increments quickly and reliably.

5.2 Why Scrum Is Suitable for This System

- **Requirements are likely to change:** support staff and customers will give new ideas once they see the first working version (for example, new notification rules or categories). Scrum welcomes change at the start of every sprint, whereas Waterfall resists it.
- **Early and frequent delivery of value:** the core features (complaint submission and ticket tracking) can be released in the first sprints, so users start benefiting early, and advanced features (SLA, reports, knowledge base) follow later.
- **User involvement:** the Product Owner (representing customers and management) and sprint reviews ensure that the system matches real helpdesk needs and is easy to use.
- **Modular product backlog:** the system naturally breaks into independent modules that can be written as user stories (e.g., “As a customer, I want to track my ticket so that I know the progress”) and prioritized by business value.
- **Risk reduction:** frequent testing and feedback expose technical or usability problems early, and integration with e-mail/SMS gateways can be tried in early sprints.
- **Team collaboration and transparency:** daily stand-ups, a visible sprint board and retrospectives improve communication, which is essential for a team building a communication-based system.
- **Suitable size and nature of project:** it is a medium-sized, web-based project with a small cross-functional team (5–8 members), which is the ideal context for Scrum.

5.3 Comparison with Other Models

Model	Suitability for This Project	Reason
Waterfall	Low	Requirements cannot be frozen in advance; user feedback arrives only at the end; changes are costly.
Spiral	Medium	Good for risk handling, but heavy documentation and cost are unnecessary for a medium-size project.
Kanban	Medium	Good for continuous support work, but lacks time-boxed iterations and planning needed for new development.
Scrum (Agile)	High (Selected)	Iterative, flexible, user-focused, delivers working software early and regularly.

Model	Suitability for This Project	Reason
DevOps	High (as support)	Used together with Scrum for automation, continuous deployment and monitoring after release.

5.4 How Scrum Is Applied in the Project

Scrum Element	Application in CCSMS
Roles	Product Owner (customer-service head), Scrum Master (facilitator), Development Team (developers, UI/UX designer, tester, DevOps engineer).
Product Backlog	Prioritised list of user stories: registration, complaint submission, ticket tracking, assignment, messaging, notifications, SLA, feedback, reports, admin.
Sprint Planning / Sprint Backlog	At the start of each 2-week sprint, the team selects the highest-priority stories and defines tasks and estimates (story points).
Daily Stand-up	15-minute meeting: what was done, what will be done, and any blockers.
Sprint Review	Demonstration of the working increment to stakeholders (agents, managers) to collect feedback.
Sprint Retrospective	Team discusses what went well and what to improve in the process.
Definition of Done	Code reviewed, unit and integration tested, security checks passed, documented and deployed to the test environment.

Suggested sprint plan:

Sprint	Goal / Main Deliverables
Sprint 0	Requirement gathering, product backlog, architecture and technology selection, UI prototypes, environment and CI/CD setup.
Sprint 1	User registration and login, role-based access, complaint submission form with ticket ID generation.
Sprint 2	Ticket list and tracking for customers, agent workbench, status updates and ticket history.
Sprint 3	In-ticket messaging with attachments, internal notes and e-mail notifications.
Sprint 4	Assignment rules, SLA engine, escalation, SMS and in-app notifications.
Sprint 5	Feedback and rating, reporting dashboards, knowledge base/FAQ.
Sprint 6	Admin console, security hardening, performance and user-acceptance testing, deployment and training.

5.5 How the Framework Supports Successful Development and Maintenance

- **Successful development:** time-boxed sprints, clear priorities, continuous testing and regular stakeholder feedback keep the project on track in terms of time, cost and quality; velocity and burn-down charts give transparent progress measurement.
- **Quality assurance:** a Definition of Done, automated tests and code reviews in every sprint prevent defects from accumulating.
- **Easy maintenance:** the same iterative cycle continues after release – new complaints categories, bug fixes and enhancements are added to the backlog and delivered in small, low-

risk releases. CI/CD pipelines, version control and monitoring reduce downtime and make rollbacks easy.

- **Knowledge sharing:** cross-functional teams, retrospectives and up-to-date documentation reduce dependency on individual developers.
- **Project management benefits (CO1):** use of user stories, story points, sprint backlog, burn-down chart, risk register and retrospectives provides effective agile project management of the software project.

6. Expected Outcomes and Limitations

6.1 Expected Benefits of the Proposed Solution

- **For customers:** quick acknowledgement, transparency, convenient 24×7 access, less effort in follow-up, and higher satisfaction and trust.
- **For support staff:** a single organized workspace, clear priorities, less manual work, ready access to customer history and canned replies, leading to faster resolution.
- **For management:** real-time visibility, data-driven decisions, reduced complaint resolution time (expected improvement of 30–50%), better SLA compliance and fewer repeat complaints.
- **For the organization:** improved reputation and customer retention, lower operating cost, better accountability through audit trails, and compliance with consumer-protection norms.

6.2 How the Solution Addresses the Identified Challenges

Identified Challenge	How It Is Addressed
Complaints lost or duplicated	Central database, unique ticket ID, duplicate detection
No visibility for customers	Customer portal with live status and history; automatic notifications
Delays and missed deadlines	Priority levels, SLA timers, reminders and auto-escalation
Scattered communication	Threaded in-ticket messaging, attachments and templates
Unbalanced workload	Rule-based assignment and workload dashboards for supervisors
No analytics for management	Dashboards and reports on volume, resolution time, SLA and satisfaction
Data security and accountability	Role-based access, encryption and audit logs
Scalability during peak loads	Modular, cloud-ready architecture with caching and queues

6.3 Potential Risks, Implementation Challenges and Limitations

Risk / Challenge	Possible Impact	Mitigation
Resistance to change from staff and customers	Low adoption of the system	Training, user-friendly design, phased roll-out and support during transition
Changing or unclear requirements	Scope creep, delays	Agile backlog management and regular reviews with the Product Owner
Failure of e-mail/SMS gateway	Missed notifications	Queue with retry, alternative provider, in-app notification as a fallback
Security breach or data leak	Loss of customer trust, legal issues	Encryption, penetration testing, access control, regular patches

Risk / Challenge	Possible Impact	Mitigation
Performance issues at peak load	Slow response or downtime	Load testing, caching, auto-scaling, monitoring
Data migration from old registers/spreadsheets	Incomplete or incorrect data	Data cleaning, validation and pilot migration
Budget and time overrun	Reduced features	Prioritised backlog (MVP first), velocity tracking, regular cost review
Spam or false complaints	Wasted effort	CAPTCHA, verification of user accounts, rate limiting

Limitations of the proposed system:

- It depends on internet connectivity and on third-party notification services.
- It cannot solve the underlying product or service defects – it only manages how complaints are handled; the quality of resolution still depends on skilled staff.
- The first release does not cover telephone (IVR) integration, advanced AI automation or multi-company (multi-tenant) support.
- Customers who are not comfortable with digital tools may still need assisted channels (phone/walk-in), which agents can log into the system on their behalf.
- Initial effort is required for training, data migration and configuration of SLA rules.

6.4 Possible Future Enhancements

- **AI chatbot / virtual assistant** to answer common questions and collect complaint details 24×7.
- **NLP-based auto-categorisation and priority prediction**, and sentiment analysis to detect angry customers early.
- **Integration with WhatsApp, social media, live chat and IVR/voice** so that all channels flow into one ticket system.
- **Native mobile apps** with push notifications.
- **Predictive analytics** to identify recurring problems and forecast ticket volume and staffing needs.
- **Self-service portal** with an enhanced knowledge base and community forum.
- **Multi-tenant and multi-language support** and integration with CRM/ERP and order-management systems.
- **Gamification and performance incentives** for agents based on customer satisfaction scores.

6.5 Conclusion

The Customer Complaint and Support Management System replaces a scattered, manual complaint process with a structured, transparent and automated helpdesk. By applying software engineering principles such as modularity, scalability, maintainability, reliability, usability and security, and by following the Agile Scrum framework supported by DevOps practices, the project can be delivered in small, valuable increments and improved continuously. The expected outcome is faster resolution, higher customer satisfaction and better decision-making for the organization, while the identified risks and limitations can be controlled through careful planning and the suggested enhancements.